

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

ваши вопросы

Copy elision

(пропуск копирования)

- оптимизация компилятора
- уменьшает количество копирований и перемещений
 - вызов `ctor`
 - вызов `dtor`

Return Value Optimization (URVO)

(unnamed, returning a prvalue)

- проверяем стандарт языка C++
 - C++17 и далее URVO обязательно
- гарантированный пропуск копирования
- компилятор конструирует экземпляр в памяти для возвращаемого значения
- не требуется `ctor`

Return Value Optimization (URVO)

(C++17 и далее)

```
class X {  
public:  
    X() = default;  
    X(const X&) = delete;  
    X(X&&) = delete;  
};
```

```
X create_x() {  
    return X();  
}
```

```
X x = create_x();
```

Named Return Value Optimization (NRVO)

- возможный пропуск копирования
- компилятор конструирует экземпляр в памяти для возвращаемого значения
- применяется к `lvalue`

Named Return Value Optimization (NRVO)

```
class X {  
public:  
    X() = default;  
    X(const X&) = delete;  
    X(X&&) = delete;  
};
```

```
X create_x() {  
    X x;  
    // ...  
    return x;  
}
```

```
X x = create_x();
```

Условия применения URVO/NRVO

- тип возвращаемого объекта (согласно сигнатуре) совпадает с типом локального (допустимо отличие на константность)
- возвращается локальный объект (а не ссылка на него или какая-то его часть)
- возвращаемый объект не должен быть `volatile` (для NRVO)

Что может мешать NRVO

- в функции несколько `return` с возвратами разных локальных объектов
- возвращаемый локальный объект ссылается на встроенный `asm`-блок
- возвращается `std::move()`

Итого

- пишите независящий от сору elision код
 - «дешевое» копирование/перемещение
 - точное управление перемещением
- URVO – стало частью языка (с C++17)
- NRVO – оптимизация, м.б. отключена
 - ключи компиляции

Оптимизация при инициализации (Temporary Materialization and Initialization)

- инициализация от `rvalue`

```
T obj = T(x, y, z);
```

- передача аргумента по значению

```
do_job(create_worker());
```

ваши вопросы

Как лучше зачитать
массив слов?

Зачитываем массив слов (без знания размера)

```
string str;  
vector<string> v;  
while (cin) {  
    cin >> str;  
    v.push_back(str);  
}
```

Зачитываем массив слов (без знания размера)

```
vector<string> v;  
while (cin) {  
    v.resize(v.size() + 1);  
    cin >> v.back();  
}
```

Зачитываем массив слов (со знанием размера)

```
cin >> n;  
v.reserve(n) ;  
for (int i =0; i < n; i += 1) {  
    cin >> v[i];  
}
```


`std::vector::push_back` (какой бывает)

```
void push_back(const T& value)
```

```
void push_back(T&& value)
```

Как лучше добавить строку литерал?

```
v.push_back(string("value")); // 1
```

```
v.push_back({"value"}); // 2
```

```
v.push_back("value"); // 3
```

Неправильно 😊

(используйте `emplace_back`)

// since C++11 - until C++17

```
template<class... Args>
```

```
void emplace_back(Args&&... args);
```

// since C++17 - until C++20

```
template<class... Args>
```

```
reference emplace_back(Args&&... args);
```

Неправильно 😊

(используйте `emplace_back`)

```
// since C++20  
template<class... Args>  
constexpr reference  
emplace_back(Args&&... args);
```

Прямая передача

(perfect forwarding)

- механизм переноса атрибутов параметров
 - `const` или не `const`
 - перемещаемость (`rvalue` или `lvalue`)
- унифицирован для (вариативных) шаблонов, фиксирующих типы и основные свойства аргументов для передаче при вызове

Экономно инициализируем поля

```
class T {  
public:  
    T(const string& v) : val_(v) {}  
    T(string&& v) : val_(move(v)) {}  
private:  
    string val_;  
};
```

А если полей несколько?

```
class T {  
public:  
    // ...  
private:  
    string v1_;  
    string v2_;  
};
```

Комбинаторный взрыв

(перегрузка всех сочетаний типов полей)

```
T(const string& v1, const string& v2)  
    : v1_(v1), v2_(v2) {}
```

```
T(const string& v1 , string&& v2)  
    : v1_(v1), v2_(move(v2)) {}
```

```
//...
```

```
T(string&& v1, string&& v2)  
    : v1_(move(v1)) , v2_(move(v2)) {}
```


Простой способ уменьшить код (copy+move)

```
T(string v1, string v2)  
  : v1_(move(v1)), v2_(move(v2)) {}
```

Продвинутый способ уменьшить код

```
template<class F, class S>  
T(F&& v1, S&& v2)  
    : v1_(forward<F>(v1))  
    , v2_(forward<S>(v2)) {  
  
}
```

ваши вопросы

Что такое T & & ?

T&& - rvalue-ссылка

- тип T – известный конкретный
- связывается только с допускающим перемещение объектом
- используется для выбора наиболее точного варианта перегрузки
- для дальнейшей передачи используем `std::move`

T & & - передающаяся ссылка (forwarding references)

- тип T — выводится
- может связываться с допускающим перемещение объектом
- для дальнейшей передачи используем `std::forward<T>`
- «правая ссылка в контексте вывода типа»

Универсальная ссылка T & & МОЖЕТ СВЯЗЫВАТЬСЯ С

- rvalue
- lvalue
- а так же
 - const и не const
 - volatile и не volatile
 - const и volatile

Универсальность ссылки T & & появляется при выводе типа

ТИП ИЗВЕСТЕН

```
void Call(D&& opt);  
  
T&& v = CreateData();
```

ТИП ВЫВОДИТСЯ

```
template<typename T>  
void Call (T&& opt);  
  
auto&& v = x;
```


T & & в шаблоне не всегда передающаяся

```
template<typename T>  
void Call(std::vector<T>&& opt);
```

T & & в шаблоне

не всегда передающаяся

```
template<typename T>
class D {
public :
    void Push(T&& src);
    // ...
}
```

Какие сложности при передаче?

```
template<typename T>  
void f(T t) {  
    T& k = t;  
}
```

```
int ii = 4;  
f<int&>(ii);
```

Сжатие/свёртка/склейка ссылок (reference collapsing)

- побеждает всегда &

& → &

&
& & → &

& &
& → &

& &
& & → & &

Прямая передача

(perfect forwarding)

- функция передает (все) параметры другой
- передаются не только значения, но и характеристики
 - `rvalue` или `lvalue`
 - `const` (если есть)
 - `volatile` (если есть)

Прямая передача

- осуществляет диспетчеризацию
- исключает ненужные копирования
- исключает использование указателей

Прямая передача

(пример – один параметр)

```
template<typename T>
void fwd(T&& param) {
    f(std::forward<T>(param)) ;
}
```

Прямая передача

(пример – произвольные параметры)

```
template<typename... Ts>
void fwd(Ts&&... params) {
    f(std::forward<Ts>(params) ...);
}
```


std::forward

- **имеет смысл использовать только в шаблонах**

```
template<class T>
T&& forward(
typename std::remove_reference<T>::type& t)
noexcept {
    return static_cast<T&&>(t);
}
```

std::make_unique

```
template<typename T, typename... Args>
unique_ptr<T> make_unique(Args&&... args) {
    return unique_ptr<T>(
        new T(std::forward<Args>(args)...) );
}
```

ваши вопросы

Как реализовать вставку в массив?

- в конец
 - увеличить размер (возможно с копированием)
 - присвоить значение
- не в конец
 - увеличить размер (возможно с копированием)
 - сдвинуть элементы
 - присвоить значение

«Наивное» копирование (с использованием цикла)

- в новый буфер

```
for (int i = 0; i < size_; i += 1) {  
    d[i] = data_[i];  
}
```

«Наивный» сдвиг (с использованием цикла)

- в существующем буфере

```
for (int i = in; i < size_ - 1; i += 1) {  
    data_[i + 1] = data_[i];  
}
```

Для POD-типов копирования/сдвиги (быстрее циклов)

- для непересекающихся областей памяти

```
void* memcpy( void* dest, const void* src,  
             std::size_t count);
```

- для произвольных областей памяти

```
void* memmove(void* dest, const void* src,  
             std::size_t count);
```

Алгоритмы копирования/сдвига (универсально и быстро)

- для непересекающихся интервалов или сдвига влево

```
OutputIt std::copy(InputIt first,  
                   InputIt last, OutputIt d_first)
```

- для сдвига вправо

```
OutputIt std::copy_backward(InputIt first,  
                             InputIt last, OutputIt d_first)
```


Заполнение

- POD-данные (C-библиотека)
- быстрее поэлементного (для простого)

```
void* std::memset(void* dest, int ch,  
    std::size_t count);
```

Заполнение

- реализация в зависимости от типа

```
// constexpr since C++20
template<class ForwardIt, class T>
void fill(ForwardIt first, ForwardIt last,
         const T& value);
```

ваши вопросы

Как написать
хороший
unit-тест?

Критерии «хорошести»

- такие же, как для обычного кода
 - выполняет свои функции
 - удобно читать
 - удобно модифицировать

Как достичь «хорошести»

- структурируйте
- используйте код повторно
- комментируйте

Как достичь «хорошести»

- такие же, как для обычного кода, тест
 - выполняет свои функции
 - удобно читать
 - удобно модифицировать

Функционал тестов

- проверяются все функции
- проверяются договоренности
- проверяются «предельные» варианты

Начальное состояние

- проверьте конструкторы
 - умолчательный
 - «ОСНОВНОЙ»

```
CHECK_EQ(0, DynArr().Size());
```

```
CHECK_EQ(15, DynArr(15).Size());
```

Особые ситуации

- моделируются
- проверяются на границах
 - есть исключение при нарушении
 - нет исключения при соблюдении
- проверяются после изменений объекта

Особые ситуации (создание)

```
CHECK_THROWS (void (DynArr (0)) ) ;
```

```
CHECK_THROWS (void (DynArr (-1)) ) ;
```

Особые ситуации (краевые)

```
DynArr ar(10);  
CHECK_THROWS(void(ar[-1]));  
CHECK_NOTHROW(void(ar[0]));  
CHECK_NOTHROW(void(ar[ar.Size()-1]));  
CHECK_THROWS(void(ar[ar.Size()]));
```

Особые ситуации (краевые после изменения)

```
ar.Resize(ar.Size() * 3);  
CHECK_THROWS(void(ar[-1]));  
CHECK_NOTHROW(void(ar[0]));  
CHECK_NOTHROW(void(ar[ar.Size()-1]));  
CHECK_THROWS(void(ar[ar.Size()]));
```

operator=

- проверяем
 - правильность для $x = x$
 - правильность для $x = y = z$
 - «совпадение» копий «по значению»
 - независимость копий

Начальное состояние для теста (генерируется)

```
DynArr ar(10);  
for (int i(0); i < ar.size(); i += 1) {  
    ar[i] = static_cast<float>(i);  
}
```

Начальное состояние для теста (копируется)

```
std::array<float> vals{0.1, 0.2, 03};  
ar.resize(std::ssize(vals));  
for (int i(0); i < std::ssize(vals); i += 1) {  
    ar[i] = vals[i];  
}
```


Проверяем operator=

- состояние ДО различное
 - «пустой»
 - буфер недостаточный
 - буфер недостаточный

```
DynArr ar_copy;
```

```
CHECK(ar.size() != ar_copy.size());
```

```
ar_copy = ar;
```

Проверяем operator=

```
good_op_eq = ar.size() == ar_copy.size();  
for (int i(0); i < ar.size(); i += 1) {  
    good_op_eq = good_op_eq &&  
        (ar[i] == doctest::Approx(ar_copy[i]));  
}  
CHECK_MESSAGE(good_op_eq,  
    "trouble operator=");
```

Как сравнить экземпляры (если нет готового `operator==`)

- написать функцию в духе `IsEqual`
 - для сравнения экземпляров
 - для сравнения с «исходными данными»

ваши вопросы